Wybrane metody Kryłowa

Ryszard Eggink

Czerwiec 2020

Abstrakt

W tej pracy przedstawione są pseudokody wybranych metod Kryłowa z książki "Iterative Methods for Sparse Linear Systems" (Saad, 2003) oraz wyjaśniona jest ich implementacja, następnie są one testowane na wybranym zadaniu eliptycznym.

1 Wstęp

Przedmiotem analizy będzie układ równań

$$Ax = b, \tag{P}$$

gdzie A to rzeczywista macierz nieosobliwa wymiaru $n \times n$. Rozwiązanie tego układu oznaczymy $x^\star \in \mathbb{R}^n$.
Metody Kryłowa rozwiązywania problemu (P) polegają na przybliżaniu rozwiązania $x^\star$ przez znalezienie $x_m \in x_0 + K_m$ przez nałożenie warunku

$$r_m = b - Ax_m \perp U_m,$$

gdzie U_m jest kolejną podprzestrzenią wymiaru m, x_0 to przybliżenie początkowe, a K_m to m-ta przestrzeń Kryłowa postaci:

$$K_m(A, r_0) \equiv span\{r_0, Ar_0, A^2r_0, \ldots, A^{m-1}r_0\}.$$

We wszystkich metodach w tej pracy wybieramy $U = AK_m$. Przedstawione w dalszych rozdziałach metody różnią się m.in. wyborem bazy K_m. W metodzie GMRES tworzymy bazę ortogonalną podczas gdy w metodach GCR, ORTHOMIN oraz ORTHODIR tworzymy bazy A^TA ortogonalne. Wszystkie metody przedstawione w pracy zbiegają do rozwiązania, jednak w tej pracy nie zajmujemy się analizą zbieżności czy własności algorytmów, a raczej ich implementacją i porównaniem na wybranym zagadnieniu testowym.

2 Wybrane metody Kryłowa

2.1 GMRES

Na początku zaimplementujemy metodę GMRES z rozróżnieniem na wybór sposobu ortogonalizacji bazy przestrzeni Kryłowa. Mianowicie dwie metody: zmodyfikowany algorytm Gramma-Schmidta oraz metodę Householdera.

Algorytm GMRES ze zmodyfikowanym G-S.

1. *Oblicz* $r_0 = b - Ax_0$, $\beta := \|r_0\|_2$ *oraz* $v_1 := \frac{r_0}{\beta}$.
2. *Dla* $j = 1, 2, \ldots, m$*:*
3. *Oblicz* $w_j := Av_j$
4. *Dla* $i = 1, \ldots, j$*:*
5. $h_{ij} := \langle w_j, v_i \rangle$
6. $w_j := w_j - h_{ij} v_i$
7. *Zakończ*
8. $h_{j+1,j} = \|w_j\|_2$. *Jeżeli* $h_{j+1,j} = 0$ *ustaw* $m := j$ *i idź do 11*
9. $v_{j+1} = \frac{w_j}{h_{j+1,j}}$
10. *Zakończ*
11. *Zdefiniuj* $(m+1) \times m$ *macierz Hessenberga* $\overline{H}_m = \{h_{ij}\}_{1 \leqslant i \leqslant m+1, 1 \leqslant j \leqslant m}$
12. *Oblicz* $y_m = \arg\min_y \left\|\beta e_1 - \overline{H}_m y\right\|_2$ *i* $x_m = x_o + V_m y_m$.

Algorytm GMRES z Householderem.

1. *Oblicz* $r_0 = b - Ax_0$, *ustaw* $z_1 = r_0$.
2. *Dla* $j = 1, 2, \ldots, m+1$*:*
3. *Oblicz jednostkowy wektor Hausholdera* w_j *taki, że*
4. $(w_j)_i = 0$, *dla* $i = 1, \ldots, j-1$
5. $(P_j z_j)_i = 0$, $i = j+1, \ldots, n$, *gdzie* $P_j = I - 2w_j w_j^T$
6. $h_{j-1} = P_j z_j$
7. $v_j = P_1 P_2 \ldots P_j e_j$
8. *Jeżeli* $j \leqslant m$ *oblicz* $z_{j+1} = P_j P_{j-1} \ldots P_1 A v_j$
9. *Zakończ*
10. *Zdefiniuj* $(m+1) \times m$ *macierz Hessenberga* $\overline{H}_m = \{h_{ij}\}_{1 \leqslant i \leqslant m+1, 1 \leqslant j \leqslant m}$
11. *Oblicz* $y_m = \arg\min_y \left\|\beta e_1 - \overline{H}_m y\right\|_2 = (\eta_1, \eta_2, \ldots, \eta_m)^T$.
12. $z = 0$
13. *Dla* $j = m, m-1, \ldots, 1$*:*
14. $z = P_j(\eta_j e_j + z)$

15. *Zakończ*
16. *Oblicz* $x_m = x_0 + z$

W kroku 12 oraz 11 odpowiednio musimy rozwiązać zadanie najmniejszych kwadratów aby obliczyć y_m. Mianowicie rozwiązać problem $y_m = \arg\min_y \left\|\beta e_1 - \overline{H}_m y\right\|_2$. Rozwiążemy ten problem w Octave za pomocą funkcji '\', mianowicie $y = \overline{H}_m \backslash b$, chociaż w książce (Saad, 2003) sugerowane jest inne podejście.

Algorytm te są niepraktyczne, gdy nasza macierz A jest dużego rozmiaru, ponieważ aby otrzymać wówczas dokładne rozwiązanie będziemy potrzebować dużego m, co wymaga dużo pamięci i mocy obliczeniowej. Wprowadzamy zatem wersję z restartem, w której aktualizujemy przybliżenie początkowe i przeprowadzamy algorytm od początku.

Algorytm GMRES z restartem.

1. *Oblicz* $r_0 = b - Ax_0$, $\beta = \|r_0\|_2$ *i ustaw* $z_1 = \frac{r_0}{\beta}$.
2. *Wygeneruj bazę przestrzeni Kryłowa przez algorytm Arnoldiego*
3. *Oblicz* $y_m = \arg\min_y \left\|\beta e_1 - \overline{H}_m y\right\|_2$ *i* $x_m = x_0 + V_m y_m$
4. *Jeżeli spełniony jest warune stopu zakończ, w przeciwnym wypadku ustaw* $x_0 := x_m$ *i idź do kroku 1*

2.2 GCR,ORTHOMIN,ORTHODIR

Metody Kryłowa są ściśle zależne od wyboru bazy przestrzeni Kryłowa. W metodzie GMRES wybieraliśmy bazę ortonormalną. W metodach w tym rozdziale wybieramy bazę tak zwaną A^TA-ortogonalną (której wektory p_i spełniają $\langle Ap_i, Ap_k\rangle = 0,\ \ dla\ \ i \neq k$).

Algorytm GCR.

1. *Oblicz* $r_0 = b - Ax_0$. *Ustaw* $p_0 = r_0$.
2. *Dla* $j = 1, 2, \ldots,$ *do zbieżności:*
3. $\quad \alpha_j = \frac{\langle r_j, Ap_j\rangle}{\langle Ap_j, Ap_j\rangle}$
4. $\quad x_{j+1} = x_j + \alpha_j p_j$
5. $\quad r_{j+1} = r_j - \alpha_j A p_j$
6. $\quad$ *Oblicz* $\beta_{ij} = -\frac{\langle Ar_{j+1}, Ap_i\rangle}{\langle Ap_i, Ap_i\rangle},\ \ dla\ \ i = 0, 1, \ldots j$
7. $\quad p_{j+1} = r_{j+1} + \sum_{i=0}^{j} \beta_{ij} p_i$
8. *Zakończ*

Oczywiście możemy trywialnie zdefiniować wersję skończoną nazywaną GCR(m), gdzie kroki wykonujemy m razy. Występuje również wersja w

której ucinamy ortogonalizację do ostatnich k wyrazów. W ten sposób powstaje algorytm znany jako ORTHOMIN(k). Zamieniamy w nim 6 i 7 linie algorytmu odpowiednio przez:

Algorytm ORTHOMIN(k).

6a. *Oblicz* $\beta_{ij} = -\frac{\langle Ar_{j+1}, Ap_i\rangle}{\langle Ap_i, Ap_i\rangle}$, *dla* $i = j-k+1, \ldots j$

7a. $p_{j+1} = r_{j+1} + \sum_{i=j-k+1}^{j} \beta_{ij} p_i$

Inna klasa algorytmów jest zdefiniowana przez obliczanie kolejnego wektora bazy p_{j+1} jako

$$p_{j+1} = Ap_j + \sum_{i=0}^{j} \beta_{ij} p_i, \tag{2.1}$$

gdzie β_{ij} są tak jak wcześniej wybrane tak, aby Ap_i były ortogonalne, tzn.

$$\beta_{ij} = -\frac{\langle A^2 p_j, Ap_i\rangle}{\langle Ap_i, Ap_i\rangle}$$

Co sprowadza się do następującego algorytmu:

Algorytm ORTHODIR.

1. *Oblicz* $r_0 = b - Ax_0$. *Ustaw* $p_0 = r_0$.
2. *Dla* $j = 1, 2, \ldots,$ *do zbieżności*
3. $\alpha_j = \frac{\langle r_j, Ap_j\rangle}{\langle Ap_j, Ap_j\rangle}$
4. $x_{j+1} = x_j + \alpha_j p_j$
5. $r_{j+1} = r_j - \alpha_j Ap_j$
6. *Oblicz* $\beta_{ij} = -\frac{\langle A^2 p_j, Ap_i\rangle}{\langle Ap_i, Ap_i\rangle}$, *dla* $i = 0, 1, \ldots j$
7. $p_{j+1} = Ap_j + \sum_{i=0}^{j} \beta_{ij} p_i$
8. *Zakończ*

Podobnie jak wcześniej algorytmy te stają się niepraktyczne, ponieważ pętla wykonywana jest do zbieżności, można to rozwiązać wprowadzając ograniczoną liczbę kroków, ale wtedy zapewne nie dostaniemy dobrego przybliżenia. Rozwiązaniem jest wprowadzenie tak jak poprzednio wersji z restartem.

Algorytm Algorytm GCR/ORTHOMIN/ORTHODIR z restartem.

1. *Oblicz* $r_0 = b - Ax_0$.
2. *Wygeneruj przybliżone rozwiązanie* x *przez wykonanie algorytmu*

GCR/ORTHOMIN/ORTHODIR, gdzie pętla po j wykonywana jest m razy

3. Oblicz $r = A - bx$, jeżeli spełniony jest warunek stopu zakończ, w przeciwnym wypadku ustaw $x_0 = x$ i przejdź do kroku 1

We wszystkich algorytmach naszym warunkiem stopu będzie ustalenie niskiego residuum, tzn. zakończ, gdy np. $r < 10^{-5}$.

3 Problem testowy

3.1 Zadanie jednowymiarowe

Będziemy testować algorytmy na problemach równań różniczkowych cząstkowych, najpierw jednowymiarowym, potem dwuwymiarowym. Za pomocą różnic skończonych można zamienić takie zadanie do układu równań liniowych. Rozpatrzmy następujące zadanie:

$$\begin{aligned} -u''(x) + bu'(x) + cu(x) &= f(x) \quad \forall x \in \Omega = [d, e] \\ u(d) &= \alpha \\ u(e) &= \beta \end{aligned} \tag{3.1}$$

dla ustalonej stałej b oraz $c > 0$, ustalonego odcinka $[d, e]$ oraz znanych wartości α, β. Będziemy przybliżać pierwszą i drugą pochodną za pomocą różnic skończonych:

$$\begin{aligned} \partial_h u(x) &= \frac{u(x+h) - u(x)}{h} \\ \partial\overline{\partial}_h u(x) &= \frac{u(x-h) - 2u(x) + u(x+h)}{h^2} \end{aligned} \tag{3.2}$$

Wprowadzamy siatkę dla $h = \frac{e-d}{N}$:

$$\overline{\Omega} = \{d + kh\}_{k=0}^{N},$$

z $\Omega_h = \{d + kh\}_{k=1}^{N-1}$ oraz $\partial\Omega_h = \{d + kh\}_{k=\{0,N} = \{d, e\}$, możemy wówczas zdefiniować następujące zadanie: znaleźć funkcję u_h określoną na siatce $\overline{\Omega}_h$ taką, że

$$\begin{cases} -\partial\overline{\partial}_h u_h(x) + b\partial_h u_h(x) + cu_h(x) = f(x) & x \in \Omega_h, \\ u(x) = g(x) & x \in \partial\Omega_h, \end{cases}$$

gdzie $g : \partial\Omega \to \mathbb{R}$ przyjmuje wartości $g(d) = \alpha$, $g(e) = \beta$. Sprowadzimy teraz powyższe równania do postaci macierzowej. Przyjmiemy następujące oznaczenia $u_k = u_h(x_k) = u_h(d+kh)$ oraz $f(x_k) = f_k$ dostajemy następujący układ równań liniowych:

$$\begin{cases} \dfrac{-u_{k-1} + 2u_k - u_{k+1}}{h^2} + \dfrac{b(u_{k+1} - u_k)}{h} + cu_k = f_k \\ u_0 = g(d) \\ u_N = g(e) \end{cases}$$

Przekształcając do postaci macierzowej otrzymujemy:

$$\frac{1}{h^2}\begin{pmatrix} 2-bh+ch^2 & bh-1 & 0 & \cdots & 0 \\ -1 & 2-bh+ch^2 & bh-1 & \cdots & 0 \\ 0 & -1 & 2-bh+ch^2 & & \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & & & bh-1 \\ 0 & 0 & -1 & & 2-bh+ch^2 \end{pmatrix} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ \vdots \\ \vdots \\ u_{N-1} \end{bmatrix} = \begin{bmatrix} f_1 + g(d) \\ f_2 \\ \vdots \\ \vdots \\ f_{N-2} \\ f_{N-1} + g(e)(1-bh) \end{bmatrix}$$

3.2 Zadanie dwuwymiarowe

Rozważmy zadanie eliptyczne dwuwymiarowe na prostokącie $\overline{\Omega} = [d, e] \times [f, g]$. Zadanie polega na znalezieniu $u \in C^2(\Omega) \cap C(\overline{\Omega})$ takiego, że

$$\begin{aligned} -\Delta u(x) + b^T \nabla u(x) + cu(x) &= f(x) \quad \forall x \in \Omega \\ u(s) &= g(s) \quad s \in \partial\Omega, \end{aligned} \tag{3.3}$$

gdzie $\Delta u(x) = \sum_{k=1}^{2} \frac{\partial^2 u}{\partial x_k^2}$, b to dany wektor, $c > 0$ jest ustaloną stałą, f to funkcja ciągła na Ω, a g to funkcja ciągła na $\partial\Omega$.

Dla ustalonego kroku $h > 0$ będziemy aproksymować pierwszą i drugą pochodną cząstkową:

$$\begin{aligned} \partial_{k,h} u(x) &= \frac{u(x + he_k) - u(x)}{h} \\ \partial\overline{\partial}_{k,h} u(x) &= \frac{u(x - he_k) - 2u(x) + u(x + he_k)}{h^2}, \quad k = 1, 2, \end{aligned} \tag{3.4}$$

gdzie e_k to $k-ty$ wektor jednostkowy. Wprowadzamy siatkę dla $h_1 = \frac{e-d}{N}$, $h_2 = \frac{g-f}{N}$:

$$\overline{\Omega}_{h_1,h_2} = \{(d+kh_1, f+lh_2)\}_{k,l=0}^{N} \tag{3.5}$$

i jej odpowiednie podzbiory $\Omega_{h_1,h_2} = \{(d+kh_1, f+lh_2)\}_{k,l=1}^{N-1} \subset \Omega$, $\partial\Omega_{h_1,h_2} = \{(d+kh_1, f+lh_2)\}_{k=\{0,N\},\, l=0,\ldots,N, l=\{0,N\},\, k=0,\ldots,N} \subset \partial\Omega$. Definiujemy następujące zadanie dyskretne: chcemy znaleźć funkcję u_{h_1,h_2} określoną na siatce $\overline{\Omega}_{h_1,h_2}$ taką, że

$$\begin{cases} -\sum\limits_{k=1,2} \partial\overline{\partial}_{k,h_k} u_{h_1,h_2}(x) + \sum\limits_{k=1,2} b_k \partial_{k,h_k} u_{h_1,h_2}(x) + c u_{h_1,h_2}(x) = f(x) \quad x \in \Omega_{h_1,h_2}, \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad u(x) = g(x) \quad x \in \partial\Omega_{h_1,h_2} \end{cases}$$

Oznaczmy dla uproszczenia zapisu $u_{k,l} = u_{h_1,h_2}(x_{k,l}) = u_{h_1,h_2}(d+kh_1, f+lh_2)$ oraz $f_{k,l} = f(x_{k,l}) = f(d+kh_1, f+lh_2)$. Otrzymujemy wówczas następujący układ liniowy.

$$\begin{cases} \dfrac{-u_{k+1,l} + 2u_{k,l} - u_{k-1,l}}{h_1^2} + \dfrac{-u_{k,l+1} + 2u_{k,l} - u_{k,l-1}}{h_2^2} + b_1 \dfrac{u_{k+1,l} - u_{k,l}}{h_1} + b_2 \dfrac{u_{k,l+1} - u_{k,l}}{h_2} + \\ + c u_{k,l} = f_{k,l}, \quad k,l = 1,\ldots,N-1 \\ u_{0,l} = g(d, f+lh_2), \quad l = 0,\ldots,N \\ u_{k,N} = g(d+kh_1, g), \quad k = 0,\ldots,N \\ u_{N,l} = g(e, f+lh_2), \quad l = 0,\ldots,N \\ u_{k,0} = g(d+kh_1, f), \quad k = 0,\ldots,N \end{cases}$$

Co można zapisać w postaci macierzowej podstawiając warunki brzegowe jako:

$$A * \begin{bmatrix} u_{1,1} \\ u_{1,2} \\ \vdots \\ u_{1,N-1} \\ u_{2,1} \\ u_{2,2} \\ \vdots \\ u_{N-1,N-2} \\ u_{N-1,N-1} \end{bmatrix} = \begin{bmatrix} f_{1,1} + \frac{g(d,f)}{h_1^2} + \frac{g(d,f)}{h_2^2} \\ f_{1,2} + \frac{g(d,f+2h_2)}{h_2^2} \\ \vdots \\ f_{1,N-1} + \frac{g(d,f+(N-1)h_2)}{h_2^2} \\ f_{2,1} + \frac{g(d+2h_1,f)}{h_1^2} \\ f_{2,2} \\ \vdots \\ f_{N-1,N-2} + \frac{g(e,f+(N-2)h_2)}{h_1^2} - b_1 \frac{g(e,f+(N-2)h_2)}{h_1} \\ f_{N-1,N-1} + g(e, f+(N-1)h_2)(\frac{1}{h_1^2} + \frac{b_1}{h_1}) + g(d+(N-1)h_1, g)(\frac{1}{h_2^2} + \frac{b_2}{h_2}) \end{bmatrix}$$

gdzie macierz A jest wymiaru $(N-1)(N-1) \times (N-1)(N-1)$ i jest dobrana tak, aby zachodziły powższe zależności.

4 Testy

4.1 Zadanie jednowymiarowe

Rozpatrzymy zadanie jednowymiarowe z $b = 1$, $c = 1$, $f(x) = 2sinx + cosx$, $d = 0$ $e = \pi$ $\alpha = 0$, $\beta = 0$. Czyli otrzymujemy następujące zadanie:

$$\begin{aligned} -u''(x) + u'(x) + u(x) &= 2\sin(x) + \cos(x) \quad \forall x \in \Omega = [0, \pi] \\ u(0) &= 0 \\ u(\pi) &= 0 \end{aligned} \tag{4.1}$$

Parametry zostały tak dobrane, aby znane było rozwiązanie $u(x) = sin(x)$. Rozwiążemy to zadanie na siatce dla $N = 100$, co sprawi, że nasza macierz A będzie miała wymiary 99×99 i porównamy wyniki uzyskane dla różnych metod. Zobaczymy dla ilu iteracji zbiegną poszczególne metody oraz jak szybko będzie spadać norma residuum (max iteracji ustalimy na 300). We wszystkich metodach ustalimy przybliżenie początkowe $x_0 = 0$ oraz za warunek stopu ustalamy normę residuum mniejszą od 10^{-5}. Dokonamy teraz porównania algorytmów dla różnego parametru m, który jak się okaże jest bardzo istotny dla metod. Zaczniemy od małego $m = 5$ (przypominam, że parametr m odpowiada za to ile iteracji wykonujemy w pierwszej pętli każdego algorytmu). W metodzie ORTHOMIN(k) ustalamy $k = 2$.

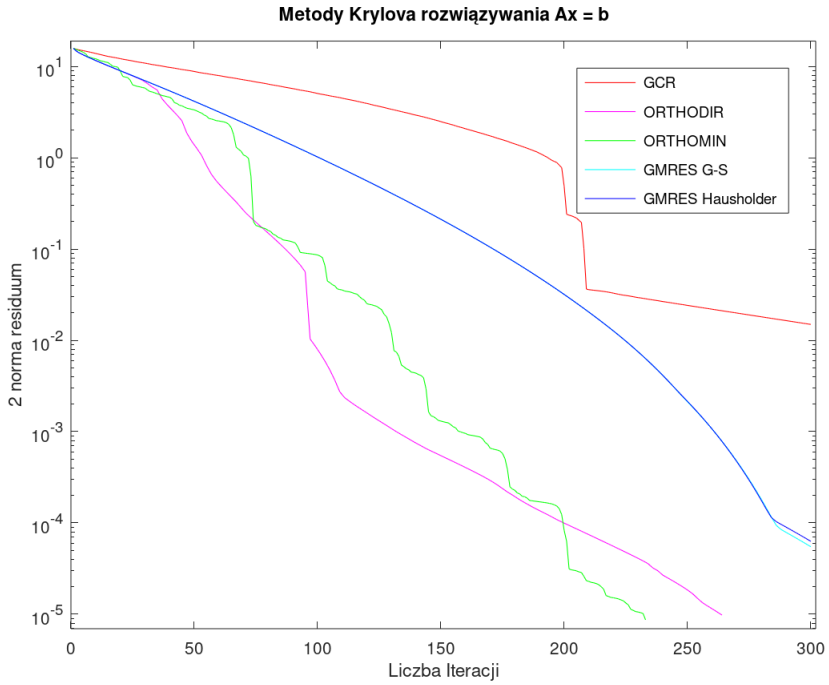

Figure 1: Wyniki dla m=5

Po pierwsze warto zauważyć, że zarówno GMRES G-S i GMRES Hausholder się pokrywają, to są algorytmy ekwiwalentne, ponieważ oba algorytmy tworzą tę samą bazę przestrzeni Kryłowa w każdej iteracji. Niestety algorytmy GMRES oraz GCR nie zbiegły do rozwiązania w 300 iteracji,dla metody GMRES residuum wyniosło wówczas 10^{-4} i dla metody GCR 10^{-1}. Metody te potrzebowałyby większej ilości iteracji aby otrzymać residuum na poziomie 10^{-5} dla wybranych parametrów. Do rozwiązania zbiegły metody ORTHODIR oraz ORTHOMIN w około 250 iteracji. Zobaczymy teraz wyniki dla tego samego zadania, gdy ustalimy $m = 25$ i $k = 10$ dla metody ORTHOMIN(k).

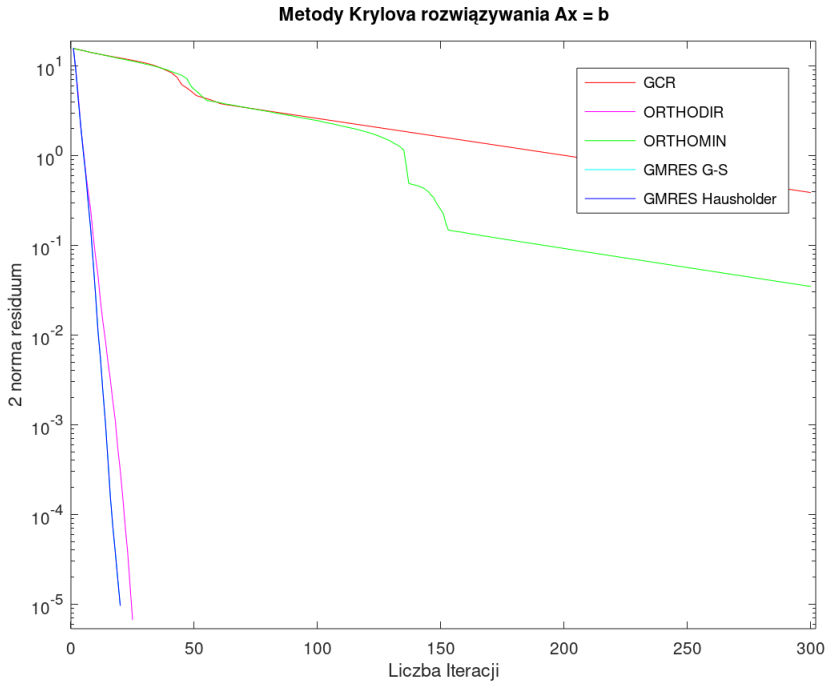

Figure 2: Wyniki dla m=25

Algorytmy GMRES oraz ORTHOMIN zbiegły do rozwiązania w około 25 iteracji. Pozostałe algorytmy nie zbiegły do rozwiązania, gdyż osiągnęły limit iteracji (300). Algorytm ORTHOMIN osiągnął po 300 iteracjach przybliżone rozwiązanie mające residuum około 10^{-1}. Najgorzej wypadł algorytm GCR.

4.2 Zadanie dwuwymiarowe

Przetestujemy teraz algorytmy na zadaniu dwuwymiarowym Laplace'a. Ustawmy $b = 0$, $c = 0$, $f \equiv 0$ oraz $\Omega = [0,1] \times [0,1]$ oraz $g(x) = sin(x_1)cos(x_1)$.

$$\begin{aligned} -\Delta u(x) &= 0 \quad \forall x \in \Omega \\ u(s) &= g(s) \quad s \in \partial\Omega, \end{aligned} \tag{4.2}$$

Wybierzemy $N = 30$. Zatem macierz A będzie miała wymiary 900×900. Tutaj musimy wybrać małe m, gdyż w przeciwnym wypadku czas obliczeniowy jest zbyt długi. Wybieramy $m = 5$ i $k = 2$ dla metody ORTHOMIN(k).

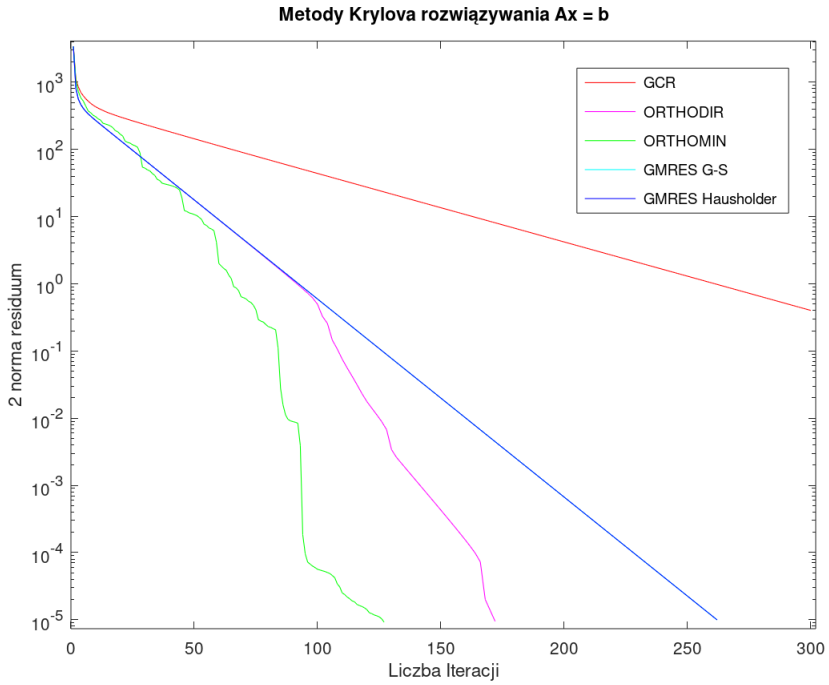

Figure 3: Wyniki dla m=5

Widzimy, że wszystkie metody oprócz GCR zbiegły do rozwiązania w mniej niż 300 iteracji. Najlepiej wypadła metoda ORTHOMIN. Wydaje się, że metoda GMRES potrzebuje większego m, aby osiągać szybsze wyniki.

References

Saad, Y. (2003). *Iterative methods for sparse linear systems* (Vol. 82). siam.